

 FH Salzburg	Fachhochschule Salzburg
	Fachrichtung: Informationstechnik & System-Management

Datenbanksysteme - Laborprotokoll
--

Titel	Library
--------------	----------------

Jahrgang	Autoren
ITS-2026	Lastowicka Susanne Lux Alexander

Bemerkungen

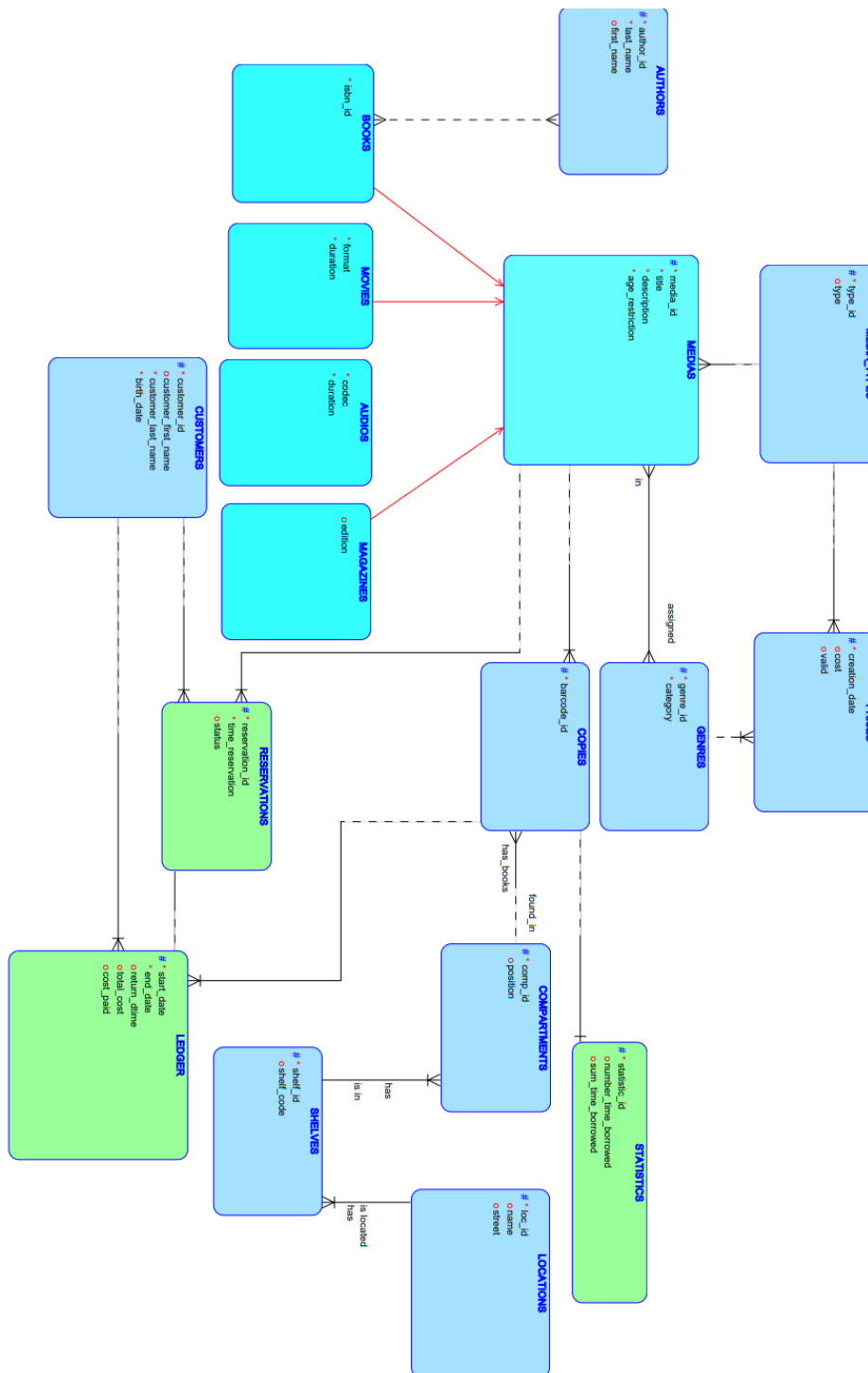
Für eine Vollständige Auflistung der Befehle und Diagramme, folgen Sie den Links.

Inhalt

1	ER-MODELL	3
1.1	Kommentare.....	4
2	DDL	4
3	INSERT STATEMENTS.....	6
3.1	Kommentare.....	7
4	TRIGGER.....	12
5	CONSTRAINTS	13
6	USE CASES.....	13
7	JOINS UND ABFRAGEN (ANWENDUNG)	17
8	MEHRBENUTZERBETRIEB.....	19
9	KONSOLEN ANWENDUNG	20
10	DATENTYPEN ANALYSIEREN UND OPTIMIEREN	24
11	STORAGE-OPTIONEN.....	25
12	ZUGRIFFSPLÄNE ANALYSIEREN UND OPTIMIEREN	26



1 ER-Modell



 FH Salzburg	Fachhochschule Salzburg
	Fachrichtung: Informationstechnik & System-Management

https://github.com/LuxAlexander/Laboruebungen/blob/main/Datenbanksysteme/Diagramme/ER_Diagram_Lastowicka_Lux_V3.pdf

1.1 Kommentare

Grün: Bewegungsdaten

Blau: Stammdaten

Türkis: Vererbung(-shierarchie)

Kann keine Reservierungen und Ausleihen geben ohne einen Kunden und Kopien. Compartments sind von Shelves abhängig, Shelves sind von Location abhängig. Statistiken sind von Kopien abhängig. Kosten sind von Genre und Medientyp abhängig.

Der Preis wird als Kostenmatrix betrachtet und der Primärschlüssel setzt sich zusammen aus type_id, genre_id und dem creation_date. Das creation_date ist deshalb Teil des Schlüssels, falls die Kosten geupdated werden.

Customer hat einen optionalen Vornamen, weil es sein kann, dass eine Familie etwas ausleiht (nur Nachname).

Bei Location reicht der Name der Library plus die Straße aus, weil nur wichtig ist, aus welcher Library man das Buch holen müsste.

Bei der Vererbung wurde eine vertikale Partitionierung, um NULL values zu vermeiden, da die Attribute sehr unterschiedlich sind und Beziehungen über den Ober-typ Media geregelt werden können.



2 DDL

```
CREATE TABLE MEDIA (  
    media_id NUMBER PRIMARY KEY,  
    title VARCHAR2(100) NOT NULL,  
    description VARCHAR2(500),  
    age_restriction NUMBER,  
    media_type_id NUMBER NOT NULL,  
    CONSTRAINT media_type_fk FOREIGN KEY (media_type_id) REFERENCES MEDIA_TYPE(media_type_id)  
);
```

```
CREATE TABLE BOOKS (  
    media_id NUMBER PRIMARY KEY,  
    isbn VARCHAR2(50) NOT NULL,  
    CONSTRAINT books_media_fk FOREIGN KEY (media_id) REFERENCES MEDIA(media_id)  
);
```

-- 6. TRANSAKTIONEN (LEDGER & RESERVATION)

```
CREATE TABLE LEDGER (  
    ledger_id NUMBER PRIMARY KEY,  
    customer_id NUMBER NOT NULL,  
    copy_id NUMBER NOT NULL,  
    start_date DATE NOT NULL,  
    end_date DATE,  
    return_date DATE,  
    total_cost NUMBER(9,2),  
    cost_paid NUMBER(9,2),  
    genre_id NUMBER, -- Teil der Kostenverknüpfung  
    media_type_id NUMBER, -- Teil der Kostenverknüpfung  
    CONSTRAINT ledger_customer_fk FOREIGN KEY (customer_id) REFERENCES CUSTOMER(customer_id),  
    CONSTRAINT ledger_copy_fk FOREIGN KEY (copy_id) REFERENCES COPY(copy_id),  
    CONSTRAINT fk_ledger_cost_matrix FOREIGN KEY (genre_id, media_type_id)  
        REFERENCES COST_EVALUATION (genre_id, media_type_id)  
);
```

V1 DDL:

https://github.com/LuxAlexander/Laboruebungen/blob/main/Datenbanksysteme/Scripte/Script-3_DDL_pretty.sql

```
CREATE TABLE PRICES (  
    MEDIA_TYPES_type_id INTEGER NOT NULL,  
    GENRES_genre_id INTEGER NOT NULL,  
    creation_date DATE NOT NULL,  
    cost NUMBER(9,2),  
    valid CHAR(1),  
    CONSTRAINT PRICES_PK PRIMARY KEY (MEDIA_TYPES_type_id, GENRES_genre_id, creation_date),  
    CONSTRAINT PRICES_GENRES_FK FOREIGN KEY (GENRES_genre_id) REFERENCES GENRES (genre_id),  
    CONSTRAINT PRICES_MEDIA_TYPES_FK FOREIGN KEY (MEDIA_TYPES_type_id) REFERENCES MEDIA_TYPES (type_id)  
);
```

 FH Salzburg	Fachhochschule Salzburg
	Fachrichtung: Informationstechnik & System-Management

```

CREATE TABLE LEDGER (
  start_date      DATE NOT NULL,
  end_date        DATE NOT NULL,
  return_dtime    DATE,
  total_cost      NUMBER(9,2),
  cost_paid       NUMBER(9,2),
  CUSTOMERS_customer_id INTEGER NOT NULL,
  COPIES_barcode_id VARCHAR2(15) NOT NULL,
  COPIES_media_id  INTEGER NOT NULL,
  CONSTRAINT LEDGER_PK PRIMARY KEY (CUSTOMERS_customer_id, COPIES_barcode_id, COPIES_media_id, start_date),
  CONSTRAINT LEDGER_CUSTOMERS_FK FOREIGN KEY (CUSTOMERS_customer_id) REFERENCES CUSTOMERS (customer_id),
  CONSTRAINT LEDGER_COPIES_FK FOREIGN KEY (COPIES_barcode_id, COPIES_media_id)
    REFERENCES COPIES (barcode_id, MEDIAS_media_id)
);

```

Version 2 DDL:

https://github.com/LuxAlexander/Laboruebungen/blob/main/Datenbanksysteme/Scripte/Script-3_2_DDL_pretty.sql

3 INSERT STATEMENTS

3.1 Kommentare

Zuerst sollten eigenständige Entitäten ohne Abhängigkeiten angelegt werden, wie z. B. Media_type, Autoren, Genre, Location und Kunden.

Anschließend werden die abhängigen Entitäten erstellt, und zwar in der Reihenfolge ihrer Abhängigkeiten. Das bedeutet beispielsweise, dass eine Location bereits existieren muss, bevor Shelves angelegt werden können. Ebenso müssen Media_type und Genre vorhanden sein, bevor eine Cost_evaluation sinnvoll definiert werden kann.

Kunden:

```
INSERT INTO CUSTOMER (first_name, last_name, birth_date) VALUES ('Alexander', 'Lux', TO_DATE('01.01.1900', 'DD.MM.YYYY'));
INSERT INTO CUSTOMER (first_name, last_name, birth_date) VALUES ('Susanne', 'Lastowicka', TO_DATE('02.02.1902', 'DD.MM.YYYY'));
```

123 CUSTOMER_ID	AZ FIRST_NAME	AZ LAST_NAME	BIRTH_DATE
1	Alexander	Lux	1900-01-01 00:00:00.000
2	Susanne	Lastowicka	1902-02-02 00:00:00.000
3	Dominik	Zach	1903-03-03 00:00:00.000
4	Max	Mustermann	2002-02-02 00:00:00.000

Version 2:

```
INSERT INTO CUSTOMERS (customer_first_name, customer_last_name, birth_date)
VALUES ('Alexander', 'Lux', TO_DATE('01.01.1950', 'DD.MM.YYYY'));
```

Medien Typ:

```
INSERT INTO FHS52423.MEDIA_TYPE
(MEDIA_TYPE_ID, NAME)
VALUES(0, 'BOOK');
```

Medien:

```
--MEDIA
INSERT INTO FHS52423.MEDIA
(TITLE, DESCRIPTION, AGE_RESTRICTION, MEDIA_TYPE_ID)
VALUES('Star Wars Episode I', 'A long time ago in a galaxy far, far away', 6, 1);
```

```
-- CONCRETE MEDIA TYPE (BOOK; MOVIE; AUDIO; MAGAZINE)
--BOOK
INSERT INTO FHS52423.BOOKS (MEDIA_ID, ISBN)
VALUES (
  (SELECT MEDIA_ID FROM FHS52423.MEDIA WHERE TITLE = '1984'),
  '9-9-9-9'
);
```

 FH Salzburg	Fachhochschule Salzburg
	Fachrichtung: Informationstechnik & System-Management

123 MEDIA_ID	AZ TITLE	AZ DESCRIPTION	123 AGE_RESTRICTION	123 MEDIA_TYPE_ID
1	Star Wars Episode I	A long time ago in a galaxy far, far away	6	1
2	1984	1984, geschrieben von 1946 bis 1948	16	0
3	Hiraeth	homesickness	0	2
4	Vogue	Druckerzeugnis	0	3
7	TEST	test	18	0

Version 2:

Medium erstellen und dem Untertypen den aktuellen Sequenzzähler mitgeben.

```
-- Insert a Book
INSERT INTO MEDIAS (title, description, age_restriction, MEDIA_TYPES_type_id)
VALUES ('1984', 'Written 1948', 16, 1);

INSERT INTO BOOKS (media_id, isbn_id)
VALUES (MEDIA_SEQ.CURRVAL, '978-3-16-148410-0');
```

Author:

```
--AUTHOR
INSERT INTO FHS52423.AUTHORS
( FIRST_NAME, LAST_NAME)
VALUES('Sapph ', '     ');
```

```
--AUTHOREN DIE ETWAS GESCHRIEBEN HABEN UND DEREN MEDIEN
--WRITTEN_BY
INSERT INTO FHS52423.WRITTEN_BY (AUTHOR_ID, MEDIA_ID)
VALUES (
  (SELECT AUTHOR_ID FROM FHS52423.AUTHORS WHERE LAST_NAME = 'Orwell' AND FIRST_NAME = 'George'),
  (SELECT MEDIA_ID FROM FHS52423.MEDIA WHERE TITLE = '1984')
);
```

Media-Genre:

```
-- Star Wars SCI-FI
INSERT INTO FHS52423.MEDIA_GENRE (MEDIA_ID, GENRE_ID)
VALUES (
  (SELECT MEDIA_ID FROM FHS52423.MEDIA WHERE TITLE = 'Star Wars Episode I'),
  (SELECT GENRE_ID FROM FHS52423.GENRE WHERE NAME = 'SCI-FI')
);
```

Version 2:

```
-- Genres
INSERT INTO GENRES (category) VALUES ('SCI-FI');
```

Standort->Regale->Fächer:

 FH Salzburg	Fachhochschule Salzburg
	Fachrichtung: Informationstechnik & System-Management

```
-- LOCATION
INSERT INTO FHS52423.LOCATION
( NAME, STREET)
VALUES('Stadtbibliothek Bischofshofen', 'Kinostraße 4');
```

```
--shelves
--BISCHOFSHOFEN
INSERT INTO FHS52423.SHELVES
( LOCATION_ID, SHELF_CODE)
VALUES( 1, 'REGAL A1');
```

```
-- COMPARTMENT
INSERT INTO FHS52423.COMPARTMENT
( SHELF_ID, "POSITION")
VALUES( 1, 'Oben');
```

Version 2:

```
--create new entry with nextval
INSERT INTO COMPARTMENTS (comp_id, SHELVES_shelf_id, SHELVES_LOCATIONS_loc_id, position)
VALUES (COMPARTMENT_SEQ.NEXTVAL, 'S1', 2, 'Top Shelf');
```

Kopien:

```
INSERT INTO FHS52423.COPY (barcode, media_id, compartment_id)
VALUES (
  'BC002',
  (SELECT media_id FROM MEDIA WHERE title = 'Star Wars Episode I'),
  2
);
```

123 COPY_ID	A-Z BARCODE	123 MEDIA_ID	123 COMPARTMENT_ID	🕒 CREATED_AT
1	BC001	3	2	2026-03-23 17:59:53.000
3	BC002	1	2	2026-03-23 18:00:52.000
4	BC003	2	1	2026-03-23 18:00:56.000
5	BC004	1	4	2026-03-23 18:00:58.000

Version 2:

```
-- Copy of Interstellar in the Sci-Fi Section
INSERT INTO COPIES (barcode_id, MEDIAS_media_id, COMPARTMENTS_comp_id, COMPARTMENTS_shelf_id, COMPARTMENTS_loc_id)
VALUES ('BC-INT-001',
  (SELECT media_id FROM MEDIAS WHERE title = 'Interstellar'),
  (SELECT comp_id FROM COMPARTMENTS WHERE position = 'Top Shelf' AND SHELVES_LOCATIONS_loc_id = 2),
  'S1', 2);
```

Preise:

```
--COST_EVAL
-- BOOK
INSERT INTO FHS52423.COST_EVALUATION
(MEDIA_TYPE_ID, COST, VALID, CREATION_DATE, GENRE_ID)
VALUES(0, 1.5, 'Y', SYSDATE, 1);
```

Version 2:(Preise)

123 MEDIA_TYPES_TYPE_ID	123 GENRES_GENRE_ID	123 CREATION_DATE	123 COST	AZ VALID
1	1	2026-04-13 18:45:12.000 GMT+	1,5	Y
1	5	2026-04-13 18:45:17.000 GMT+	1,2	Y
1	3	2026-04-13 18:45:19.000 GMT+	2	Y
2	1	2026-04-13 18:45:24.000 GMT+	3,5	Y
2	6	2026-04-13 18:45:26.000 GMT+	3	Y
3	2	2026-04-13 18:45:27.000 GMT+	1	Y
4	4	2026-04-13 18:45:30.000 GMT+	0,8	Y

Ausleihen:

```
--LEDGER
INSERT INTO FHS52423.LEDGER
(CUSTOMER_ID, COPY_ID, START_DATE, END_DATE, TOTAL_COST, MEDIA_TYPE_ID, GENRE_ID)
VALUES (
  (SELECT customer_id FROM CUSTOMER WHERE last_name = 'Lux'),
  (SELECT copy_id FROM COPY WHERE barcode = 'BC004'),
  SYSDATE - 10,
  SYSDATE - 5,
  (SELECT MAX(COST) FROM COST_EVALUATION WHERE media_type_id = 0),
  0,
  (SELECT genre_id FROM COST_EVALUATION
   WHERE media_type_id = 0 AND cost = (SELECT MAX(cost) FROM COST_EVALUATION WHERE media_type_id = 0)
   AND ROWNUM = 1)
);
```

123 ID	AZ KUNDE	AZ MEDIUM_TITEL	AZ BARCODE	AZ TYP	AZ GENRE	AZ AUSGELIEHEN	AZ RUECKGABE_SOLL	AZ PREIS
2	Dominik Zach	Hiraeth	BC001	AUDIO	Dystopie	14.03.2026	18.03.2026	10 €
1	Alexander Lux	Star Wars Episode I	BC004	MOVIE	SCI-FI	13.03.2026	18.03.2026	1,5 €

Version 2:

	START_DATE	END_DATE	RETURN_DATE	123 TOTAL_COST	123 COST_PAID	123 CUSTOMERS_CUSTOMER_ID	AZ COPIES_BARCODE_ID	123 COPIES_MEDIA_ID
1	2026-04-11 17:56:35.000 GH	2026-04-18 17:56:35.000	2026-04-18 17:57:35.000	4,5	[NULL]	3	BC-INT-001	4
2	2026-04-11 00:00:00.000 GH	2026-04-18 00:00:00.000	[NULL]	4,5	[NULL]	3	BC-INT-001	4
3	2026-04-11 00:00:00.000 GH	2026-04-18 00:00:00.000	[NULL]	1,2	[NULL]	1	BC-HOBBIT-01	3

```
● INSERT INTO LEDGER (start_date, end_date, CUSTOMERS_customer_id, COPIES_barcode_id, COPIES_media_id, total_cost)
SELECT
  TRUNC(SYSDATE) - 2,
  TRUNC(SYSDATE) + 5,
  (SELECT customer_id
   FROM CUSTOMERS
   WHERE customer_last_name = 'Lux'
   FETCH FIRST 1 ROW ONLY),
  'BC-HOBBIT-01',
  (SELECT m.media_id FROM MEDIAS m WHERE m.title = 'The Hobbit'),
  (
    SELECT MAX(p.cost)
    FROM PRICES p
    JOIN MEDIA_GENRE mg
      ON p.GENRES_genre_id = mg.GENRES_genre_id
   WHERE mg.MEDIAS_media_id = m.media_id
   AND p.MEDIA_TYPES_type_id = m.MEDIA_TYPES_type_id
   AND p.valid = 'Y'
  )
);
```

 FH Salzburg	Fachhochschule Salzburg
	Fachrichtung: Informationstechnik & System-Management

Reservierungen:

```

INSERT INTO FHS52423.RESERVATION (
  customer_id,
  copy_id,
  reservation_date,
  status
)
VALUES (
  (SELECT customer_id FROM CUSTOMER WHERE last_name = 'Lastowicka'),
  (SELECT copy_id FROM COPY WHERE barcode = 'BC004'),
  SYSDATE,
  'AKTIV'
);

```

Reservierung besitzen den Fremdschlüssel, um Null werte zu vermeiden, weil Reservierungen optional sind.

123 RESERVATION_ID	TIME_RESERVATION	123 CUSTOMER_ID	123 MEDIA_ID	A2 STATUS	A2 BARCODE_ID	START_DATE
3	2026-04-13 17:58:03.000	1	3	PENDING	BC-INT-001	2026-04-11 00:00:00.000

Statistica:

123 STATISTIC_ID	123 NUMBER_TIME_BORROWED	123 SUM_TIME_BORROWED	A2 COPIES_BARCODE_ID	123 COPIES_MEDIAS_MEDIA_ID
4	1	2	BC-HOBBIT-01	3
5	3	11	BC-INT-001	4

```

-- INSERT INTO STATISTICS (
  number_time_borrowed,
  sum_time_borrowed,
  COPIES_barcode_id,
  COPIES_medias_media_id
)
SELECT
  COUNT(*) as number_time_borrowed,
  SUM(TRUNC(NVL(return_dtime, SYSDATE)) - TRUNC(start_date)) as sum_time_borrowed,
  COPIES_barcode_id,
  COPIES_media_id
FROM LEDGER
GROUP BY COPIES_barcode_id, COPIES_media_id;

```

Liste der ersten Inserts:

https://github.com/LuxAlexander/Laboruebungen/blob/main/Datenbanksysteme/Scripte/Script-2_INSERTS.sql

Liste aller neuen Inserts:

https://github.com/LuxAlexander/Laboruebungen/blob/main/Datenbanksysteme/Scripte/Script-2_2_INSERTS.sql

4 TRIGGER

Mit dem Gedanken, dass sich nicht-natürliche Schlüssel automatisch erhöhen bei Eingabe von Objekten, haben wir Auto-Inkrement eingebaut.

```
-- Auto-Increment Triggers
CREATE OR REPLACE TRIGGER media_bi BEFORE INSERT ON MEDIA FOR EACH ROW BEGIN
  IF :NEW.media_id IS NULL THEN SELECT media_seq.NEXTVAL INTO :NEW.media_id FROM dual; END IF; END;
/
CREATE OR REPLACE TRIGGER author_bi BEFORE INSERT ON AUTHORS FOR EACH ROW BEGIN
  IF :NEW.author_id IS NULL THEN SELECT author_seq.NEXTVAL INTO :NEW.author_id FROM dual; END IF; END;
/
CREATE OR REPLACE TRIGGER customer_bi BEFORE INSERT ON CUSTOMER FOR EACH ROW BEGIN
```

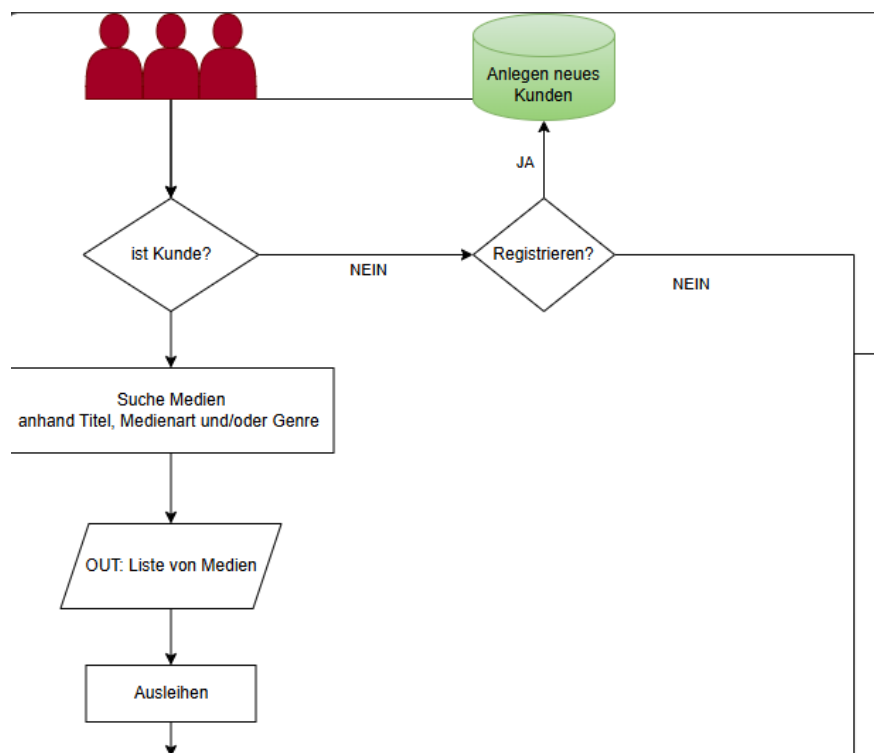
5 CONSTRAINTS

```
CREATE TABLE COPY (
  copy_id NUMBER PRIMARY KEY,
  barcode VARCHAR2(50) UNIQUE NOT NULL,
  media_id NUMBER NOT NULL,
  compartment_id NUMBER NOT NULL,
  created_at DATE DEFAULT SYSDATE,
  CONSTRAINT copy_media_fk FOREIGN KEY (media_id) REFERENCES MEDIA(media_id),
  CONSTRAINT copy_compartment_fk FOREIGN KEY (compartment_id) REFERENCES COMPARTMENT(compartment_id)
);
```

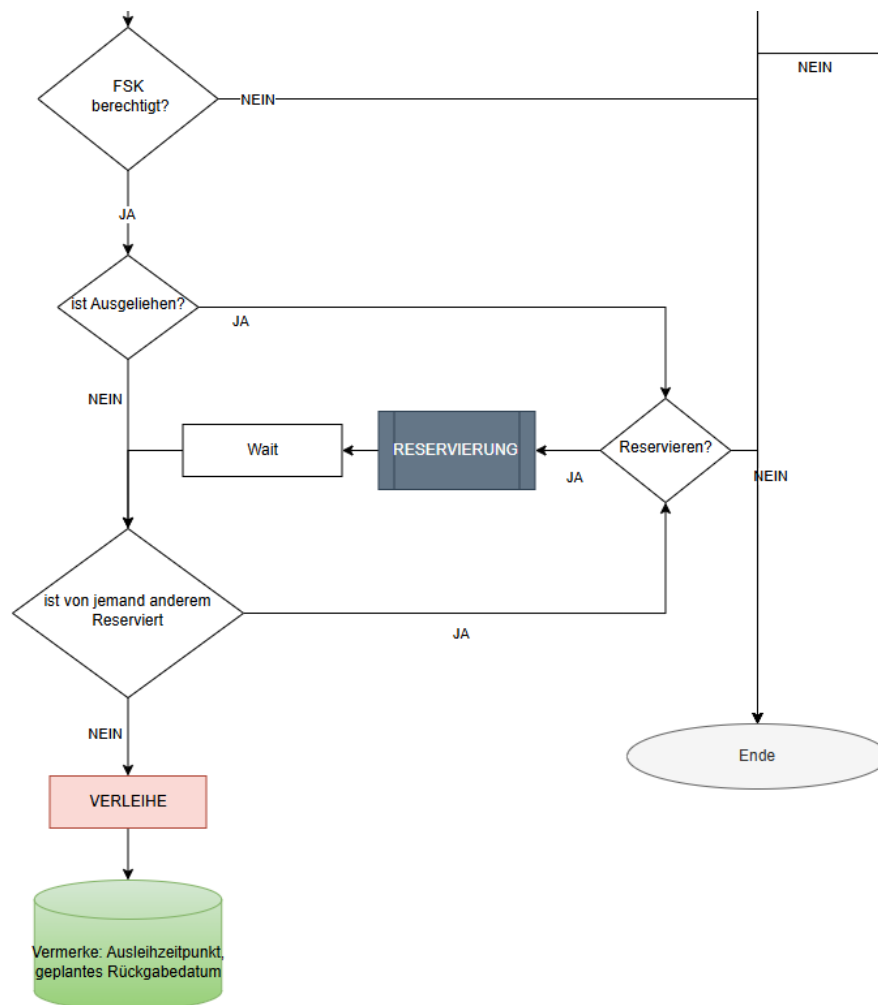
6 Use Cases

Ausleihen

Möchte ein Kunde etwas ausleihen, muss er zuerst ins System eingetragen werden (Mindestens mit Nachnamen und Geburtsdatum). Danach kann er durch Eingabe von entweder Titel, Medienart und/oder Genre nach Medien suchen, diese werden ausgegeben.



Beim Ausleihen wird zuerst überprüft, ob man alt genug ist. Dann wird geprüft, ob bereits alle Exemplare verliehen sind, wenn ja besteht die Möglichkeit es zu reservieren. Danach wird die Reservierungsliste durchgegangen, bis man an der Reihe ist. Die älteste Reservierung wird ausgewählt. Dem Kunden wird das Exemplar zugewiesen. In der Datenbank wird gespeichert, wer, was, wann und bis wann ausleiht (Kunde, Medium, Zeitraum).



Reservieren

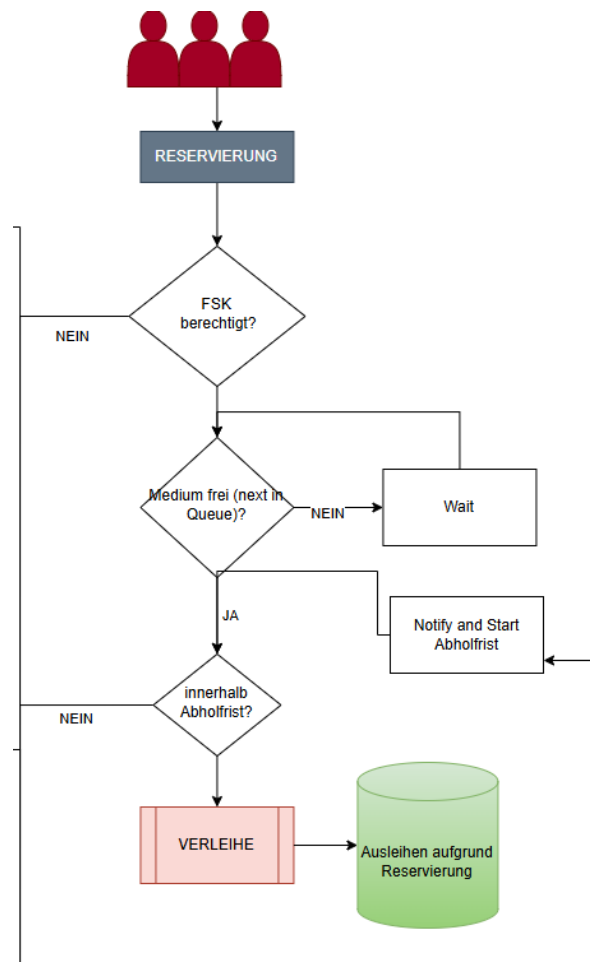
Möchte ein Kunde etwas Reservieren, wird zuerst überprüft, ob er alt genug für das Medium ist. Ist das der Fall wird in der Tabelle vermerkt: wer, was, wann reserviert (Kunde, Medium, Datum), weiteres kommt er in eine Warteschlange (Reihenfolge der Kunden, nach Reservierungsdatum aufsteigend sortiert), bis das nächste Exemplar für den Kunden frei ist. Er kann sich seinen Artikel, innerhalb von 3 Tagen abholen, ansonsten verfällt die Reservierung. Beim Ausleihen wird in der Reservierungstabelle der Verweis auf die Ausleihe gespeichert.



FH Salzburg

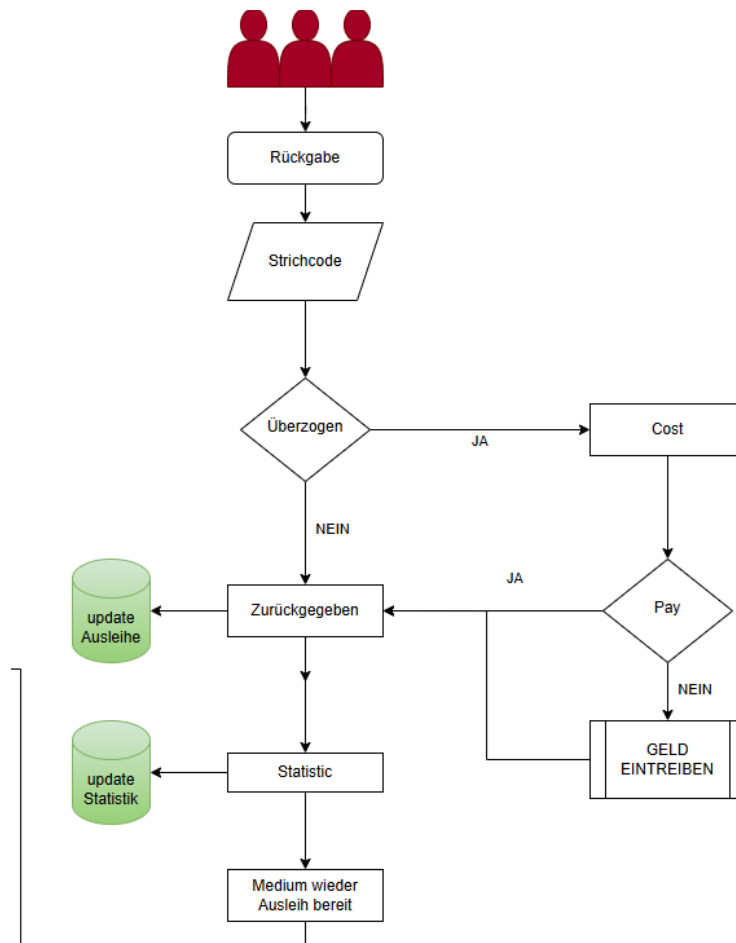
Fachhochschule Salzburg

Fachrichtung: Informationstechnik & System-Management



Rückgabe

Bei der Rückgabe eines Buches wird aufgrund des Strichcodes überprüft, um welches Exemplar es sich handelt. Dann wird aufgrund dem im Ledger festgelegten start_date überprüft, ob die Rückgabefrist eingehalten wurde oder nicht. Bei Nichteinhaltung wird der Preis für das Überziehen berechnet und der Kunde muss diesen zahlen. Wird nicht gezahlt (was nicht explizit als Fall genannt wird), wird das Geld auf andere Art angefordert. Dann wird das Buch zurückgegeben, die STATISTICS updaten für dieses Exemplar die Zeit, und die Anzahl der Ausleihen wird um 1 erhöht. Der LEDGER wird auch geupdatet (return time, cost, cost paid), dadurch kann die Kopie auch direkt wieder ausgeliehen werden.



https://github.com/LuxAlexander/Laboruebungen/blob/main/Datenbanksysteme/Diagramme/Use_Cases.drawio.svg



7 JOINS und Abfragen (Anwendung)

Rückgabe (Ausschnitt)

```
-- =====
-- RÜCKGABE
-- =====

-- 1. Medium/Kopie identifizieren (Schritt: "Strichcode")
-- Prüft, ob der Barcode existiert und holt die Stammdaten
SELECT c.*, m.TITLE
FROM COPIES c
JOIN MEDIAS m ON c.Medias_MEDIA_ID = m.MEDIA_ID
WHERE c.BARCODE_ID = :barcode
FOR UPDATE WAIT 5;

-- 2. Kosten berechnen & im Ledger speichern (Schritt: "Überzogen?" & "Cost")
-- Dieser Befehl berechnet die Gebühr und schreibt sie in TOTAL_COST.
-- GREATEST(0, ...) verhindert negative Kosten bei rechtzeitiger Abgabe.
-- 'cost'-Wert aus PRICES, der aktuell 'valid' ist.
-- Dieser Wert wird mit der Anzahl der Tage Multipliziert.
UPDATE LEDGER l
SET l.TOTAL_COST = GREATEST(0,
    ROUND(
        (SYSDATE - l.END_DATE) * (
            SELECT p.cost
            FROM PRICES p
            -- Verknüpfung über MEDIAS, um den passenden Preis für die Kopie zu finden
            JOIN MEDIAS m ON (m.MEDIA_TYPES_TYPE_ID = p.type_id AND m.genre_id = p.genre_id)
            JOIN COPIES c ON (c.Medias_MEDIA_ID = m.MEDIA_ID)
            WHERE c.BARCODE_ID = l.COPIES_BARCODE_ID
            AND p.valid <= SYSDATE -- Preis muss bereits gültig sein
            ORDER BY p.valid DESC -- Das aktuellste Datum zuerst
            FETCH FIRST 1 ROW ONLY
        ), 2)
    )
WHERE l.COPIES_BARCODE_ID = :barcode
AND l.RETURN_DTIME IS NULL;
```

Kundenverwaltung

```
-- =====
-- KUNDENVERWALTUNG & SUCHE
-- =====

-- Prüfen, ob Kunde existiert (ist Kunde?)
SELECT * FROM CUSTOMERS WHERE CUSTOMER_LAST_NAME = :last_name AND BIRTH_DATE = :birth_d;

-- Neuen Kunden anlegen (Anlegen neues Kunden)
INSERT INTO CUSTOMERS (CUSTOMER_FIRST_NAME, CUSTOMER_LAST_NAME, BIRTH_DATE)
VALUES (:first_n, :last_n, :birth_d);

-- Suche Medien (Titel, Genre oder Typ)
SELECT m.*, g.CATEGORY, mt.TYPE
FROM MEDIAS m
JOIN MEDIA_TYPES mt ON m.MEDIA_TYPES_TYPE_ID = mt.TYPE_ID
JOIN GENRES g ON m.MEDIA_ID = g.GENRE_ID
WHERE m.TITLE LIKE %:search% OR g.CATEGORY = :genre OR mt.TYPE = :type;
```

Verleihen



```
-- =====
-- VERFÜGBARKEIT & VERLEIHE
-- =====

-- FSK Prüfung (FSK berechtigt?)
-- Rechnet das Alter in Jahren aus und vergleicht mit age_restriction
SELECT * FROM MEDIAS m
WHERE m.MEDIA_ID = :media_id
AND m.age_restriction <= FLOOR(MONTHS_BETWEEN(SYSDATE, (SELECT birth_date FROM CUSTOMERS WHERE customer_id = :c_id)) / 12);

-- Prüfen, ob eine Kopie aktuell frei ist (ist Ausgeliehen?)
-- Ein Medium ist frei, wenn es eine Kopie gibt, die nicht in einem offenen Ledger-Eintrag steht
SELECT * FROM COPIES c
WHERE c.Medias_MEDIA_ID = :media_id
AND c.BARCODE_ID NOT IN (SELECT COPIES_BARCODE_ID FROM LEDGER WHERE RETURN_DTIME IS NULL)
FOR UPDATE WAIT 5;

-- Verleihen: Neuen Eintrag im Ledger anlegen
INSERT INTO LEDGER (START_DATE, END_DATE, TOTAL_COST, COST_PAID, CUSTOMERS_CUSTOMER_ID, COPIES_BARCODE_ID)
VALUES (SYSDATE, SYSDATE + 21, (SELECT cost FROM PRICES WHERE creation_date = (SELECT MAX(creation_date) FROM PRICES)), 0, :c_id, :barcode);
```

Reservierung

```
-- =====
-- RESERVIERUNG
-- =====

-- Neue Reservierung anlegen (Warteschlange)
INSERT INTO RESERVATIONS (TIME_RESERVATION, STATUS, CUSTOMER_ID, MEDIA_ID)
VALUES (SYSDATE, 'PENDING', :c_id, :media_id);

-- Prüfen: Wer ist der nächste in der Schlange? (next in Queue?)
SELECT * FROM RESERVATIONS
WHERE MEDIA_ID = :media_id AND STATUS = 'PENDING'
ORDER BY TIME_RESERVATION ASC
FOR UPDATE SKIP LOCKED -- Überspringt Datensätze, die bereits von anderen in Bearbeitung sind
FETCH FIRST 1 ROW ONLY;

-- Reservierung abschließen (Wenn abgeholt wird)
UPDATE RESERVATIONS
SET STATUS = 'DONE'
WHERE RESERVATION_ID = :res_id;
```

Ist der Kunde, der am längsten in der Reservierungswarteliste ist, an der Reihe, weil ein Exemplar frei geworden ist, und holt dieses nicht innerhalb der 3 Tage ab, verfällt die Reservierung. Das wird rein über die Software geregelt.

https://github.com/LuxAlexander/Laboruebungen/blob/main/Datenbanksysteme/Scripte/Script-2_Anwendung.sql

 FH Salzburg	Fachhochschule Salzburg
	Fachrichtung: Informationstechnik & System-Management

8 Mehrbenutzerbetrieb

Für den Mehrbenutzerbetrieb wird “Read Committed” verwendet und “Auto Commit” wird deaktiviert. Kritische Blöcke, bei denen es zu Anomalien kommen könnte, werden in erste Linie mit “FOR UPDATE” behandelt. Zum Beispiel beim automatischen Auswählen des nächsten Customers, der das Medium reserviert hat.

```

next_in_reservation_queue: "SELECT * FROM RESERVATIONS
  WHERE RESERVATION_ID = (
    SELECT RESERVATION_ID FROM (
      SELECT RESERVATION_ID FROM RESERVATIONS
      WHERE MEDIA_ID = :media_id AND STATUS = 'READY'
      ORDER BY TIME_RESERVATION ASC
    )
    WHERE ROWNUM = 1
  )
  FOR UPDATE SKIP LOCKED"

```

9 Konsolen Anwendung

Ablauf:

Main(): Innerhalb einer Schleife, muss man sich zuerst einzuloggen oder registrieren (Nachname + Geburtsdatum). Danach sieht man alle Reservierungen, die ausgelaufen sind. Man hat dazu die Wahl sich im Menü zwischen mehreren Optionen zu entscheiden (Medien Auswahl, Medien Zurückgeben, Reservierungen Checken, Zeit zu seinen Reservierungen hinzufügen).

Transaction mit commit: Das Medium wird reserviert. Das Update wird zur Datenbank committed. Tritt ein Error auf, kommt es zu einem Rollback.

Transaction ohne commit: Für den FSK Check wird kein commit benötigt, da nur eine Überprüfung des Alters stattfindet.

```

500     with open(file) as f: # load the yaml file
501         data = yaml.load(f, Loader=SafeLoader)
502     # Read user input in a loop
503     while True:
504         # If no one is logged in, show the Auth Menu
505         if not customer:
506             print("\nWelcome to the Library!")
507             choice = input("1. Login\n2. Create Account\nq. Quit\nSelection: ")
508
509             if choice == '1':
510                 lname = input("Lastname: ")
511                 bdate = input("Date of Birth (DD.MM.YYYY): ")
512                 customer = login(data, lname, bdate)
513             elif choice == '2':
514                 customer = registration(data)
515             elif choice == 'q':
516                 break
517
518         # If someone IS logged in, show the Library Menu
519         else:
520
521
522             print(f"\n--- Logged in as {customer[1]} ---")
523             check_reservations(data, customer[0])
524             print("1. Search for Media")
525             print("2. Return Media")
526             print("3. Check Reservations")
527             print("q. Logout")

```



```
339 def reservation(data: str, media_id, customer_id) -> None:
340     try:
341         db.conn.begin() # start transaction (usually not needed, but still best practice)
342
343         # we need a cursor to execute queries
344         cursor = db.conn.cursor()
345
346         # execute the login query
347         cursor.execute(data['create_reservation'], {"c_id": customer_id, "media_id": media_id })
348         # Commit transaction
349         db.conn.commit()
350
351         print(f"Successfully reserved the Media!\nYou will be notified when it is available.")
352
353     except oracledb.DatabaseError as e:
354         print("An error occurred executing the query:", e)
355         print_exc() # print stack trace
356         db.conn.rollback() # rollback any changes in case of an error
357         raise e
358     finally:
359         cursor.close() # always close the cursor when done
360
create_reservation: "INSERT INTO RESERVATIONS (TIME_RESERVATION, STATUS, CUSTOMER_ID, MEDIA_ID)
VALUES (SYSDATE, 'PENDING', :c_id, :media_id)"

167 def fsk_check(data: str, media_id, customer_id) -> None:
168     try:
169         db.conn.begin() # start transaction (usually not needed, but still best practice)
170
171         # we need a cursor to execute queries
172         cursor = db.conn.cursor()
173
174         # execute the login query
175         cursor.execute(data['fsk_check'], {"media_id": media_id, "c_id": customer_id})
176         allowed_media = cursor.fetchone()
177
178         if allowed_media:
179             print("Access granted! You are old enough.")
180             copy_available(data, media_id, customer_id)
181         else:
182             print("Access denied: You do not meet the age requirement for this media.")
183
184     except oracledb.DatabaseError as e:
185         print("An error occurred executing the query:", e)
186         print_exc() # print stack trace
187         db.conn.rollback() # rollback any changes in case of an error
188         raise e
189     finally:
190         cursor.close() # always close the cursor when done
fsk_check: "SELECT * FROM MEDIAS m
WHERE m.MEDIA_ID = :media_id
AND m.age_restriction <= FLOOR(MONTHS_BETWEEN(SYSDATE, (SELECT birth_date FROM CUSTOMERS WHERE customer_id = :c_id)) / 12)"
```

Screenshots:

```

Selection: 1Traceback (most recent call last):
  2. Star Wars Episode I | Fantasy | MOVIE
-----
Enter the number to borrow/reserve (or 'b' to go back): 1
You selected: Star Wars Episode I
Access granted! You are old enough.
The reservation queue for this media is empty.
Successfully borrowed the Media!
Please return it in 21 Days.
Media borrowed successfully.

--- Logged in as Alexander ---
No open reservations.
1. Search for Media
2. Return Media
3. Check Reservations
q. Logout
Selection: []

PS C:\Users\Alex\source\GruppenArbeiten\Susi\Laboruebungen\Datenbanksysteme\dbspython_library> poetry run python main.py
*** MEDIA SEARCH ***
Leave a field blank if you don't want to filter by it.
Search Title: star
Search Genre:
Search Media-Type:

Found 2 matches:
-----
1. Star Wars Episode I | SCI-FI | MOVIE
2. Star Wars Episode I | Fantasy | MOVIE
-----
Enter the number to borrow/reserve (or 'b' to go back): 1
You selected: Star Wars Episode I
Access granted! You are old enough.
There is no available copy of this media currently. Want to reserve it? (y/n)
Selection:

```

Kunde A nimmt eine Kopie, dann wird Kunde B die Wahl gegeben das Medium zu Reservieren.

```

Successfully reserved the Media!
You will be notified when it is available.

--- Logged in as Lux ---
Open Reservations:
RESERVATION_ID | TIME_RESERVATION | CUSTOMER_ID | MEDIA_ID | STATUS | BARCODE_ID | START_DATE
-----
43 | 2026/05/11 | 21 | 2 | PENDING | NULL | NULL

```

Kunde sieht seine Reservierungen und deren Status.

```

PS C:\Users\Alex\source\GruppenArbeiten\Susi\Laboruebungen\Datenbanksysteme\dbspython_library> poetry run python main.py
Welcome to the Library!
1. Login
2. Create Account
q. Quit
Selection: 1
Lastname: Lux
Date of Birth (DD.MM.YYYY): 01.01.1950
Success! Welcome Lux.

--- Logged in as Alexander ---
No open reservations.
1. Search for Media
2. Return Media
3. Check Reservations
q. Logout
Selection: []

PS C:\Users\Alex\source\GruppenArbeiten\Susi\Laboruebungen\Datenbanksysteme> poetry run python main.py
-----
1. Star Wars Episode I | SCI-FI | MOVIE
2. Star Wars Episode I | Fantasy | MOVIE
-----
Enter the number to borrow/reserve (or 'b' to go back): 1
You selected: Star Wars Episode I
Access granted! You are old enough.
You are next in the reservation queue for this media. You can borrow it.
Successfully borrowed the Media!
Please return it in 21 Days.
Media pickup confirmed successfully.
Media borrowed successfully.

--- Logged in as Lux ---
No open reservations.
1. Search for Media
2. Return Media

```

Wenn man als nächstes dran ist, darf man es Ausleihen.

123 RESERVATION_ID	TIME_RESERVATION	123 CUSTOMER_ID	123 MEDIA_ID	AZ STATUS	AZ BARCODE_ID	START_DATE
43	2026-05-11 18:11:20.000	21	2	DONE	MV-STAR-01	2026-05-11 00:00:00.000

Leiht man sich aufgrund einer Reservierung aus, wird die Tabelle geupdated und der Status von PENDING über READY zu DONE.

	123 RESERVATION_ID	TIME_RESERVATION	123 CUSTOMER_ID	123 MEDIA_ID	AZ STATUS	AZ BARCODE_ID	START_DATE
1	61	2026-05-11 18:12:20.000	21	1	PENDING	[NULL]	[NULL]
2	62	2026-04-11 18:12:20.000	21	1	READY	[NULL]	[NULL]

	123 RESERVATION_ID	TIME_RESERVATION	123 CUSTOMER_ID	123 MEDIA_ID	AZ STATUS	AZ BARCODE_ID	START_DATE
1	62	2026-04-11 18:12:20.000	21	1	FAILED	[NULL]	[NULL]
2	61	2026-05-11 18:12:20.000	21	1	READY	[NULL]	[NULL]

Wenn der Kunde seine Frist von drei Tagen verstreichen lässt, wird der Status auf FAILED gesetzt und der nächste zu READY.

```

4. More Reservation Time
q. Logout
Selection: 4
Successfully increased reservation time for all your reservations by 2 days!
No failed pickups

```

Der Kunde kann seine Reservierung verlängern.

	123 STATISTIC_ID	123 NUMBER_TIME_BORROWED	123 SUM_TIME_BORROWED	A-Z COPIES_BARCODE_ID	123 COPIES_MEDIAS_MEDIA_ID
1	4	1	2	BC-HOBBIT-01	3
2	5	3	11	BC-INT-001	4

	123 STATISTIC_ID	123 NUMBER_TIME_BORROWED	123 SUM_TIME_BORROWED	A-Z COPIES_BARCODE_ID	123 COPIES_MEDIAS_MEDIA_ID
1	4	2	3	BC-HOBBIT-01	3
2	5	3	11	BC-INT-001	4
3	21	1	1	MV-STAR-01	2

Wenn ein Kunde eine Kopie zurückgibt, wird in der Statistik für diese spezifische Kopie die Spalte NUMBER_TIME_BORROWED um 1 erhöht und SUM_TIME_BORROWED um die Anzahl der Tage erhöht. Existiert für diese Kopie noch keine Statistik, wird eine neue Zeile erstellt und die STATISTIC_ID wird automatisch zugeordnet.

https://github.com/LuxAlexander/Laboruebungen/blob/main/Datenbanksysteme/dbspython_library/main.py

https://github.com/LuxAlexander/Laboruebungen/blob/main/Datenbanksysteme/dbspython_library/my_sql_file.yaml

10 Datentypen analysieren und optimieren

Änderungen:

Aus einem Date Datentype für Film und Audio länge wurde ein Integer mit 4 Ziffern gewählt (Dauer in Minuten).

Die gewählte Größe mit Varchar2(50) der ISBN wurde auf 17 reduziert.

Damit längere Beschreibungen effizient möglich verwendet nun Description, CLOB.

Bei den DDL Statements werden die unendlichen NUMBER(*,0) angaben auf z.B. NUMBER(9,0) beschränkt.

```
--Datentypen analysieren und optimieren
-- Wechsel von DATE zu Minuten-Integer, Zwischen Puffer für Movie (New Duration=Old, delete Old)
ALTER TABLE "FHS52423"."MOVIES" MODIFY ("DURATION" NUMBER(4,0));
ALTER TABLE "FHS52423"."AUDIOS" MODIFY ("DURATION" NUMBER(4,0));

-- Optimierung der ISBN-Spalte von 50 auf 17
ALTER TABLE "FHS52423"."BOOKS" MODIFY ("ISBN_ID" VARCHAR2(17));

-- Erhöhung des Textpuffers für Medienbeschreibungen (Alternativ CLOB), same as movie
ALTER TABLE "FHS52423"."MEDIAS" MODIFY (
  "DESCRIPTION" CLOB) LOB ("DESCRIPTION") STORE AS MEDIAS_DESC_LOB (
  ENABLE STORAGE IN ROW
  CHUNK 8192
  RETENTION
  CACHE
);
-- Number grÖße einschränken, wenn Tabellen neu erstellt werden. (Wasn't used)
ALTER TABLE "FHS52423"."AUTHORS" MODIFY ("AUTHOR_ID" NUMBER(9,0));
ALTER TABLE "FHS52423"."CUSTOMERS" MODIFY ("CUSTOMER_ID" NUMBER(9,0));
ALTER TABLE "FHS52423"."GENRES" MODIFY ("GENRE_ID" NUMBER(4,0));
ALTER TABLE "FHS52423"."LOCATIONS" MODIFY ("LOC_ID" NUMBER(6,0));
ALTER TABLE "FHS52423"."MEDIA_TYPES" MODIFY ("TYPE_ID" NUMBER(3,0));
ALTER TABLE "FHS52423"."MEDIAS" MODIFY ("MEDIA_ID" NUMBER(9,0));
```

11 Storage-Optionen

Zwei der größten Tabellen sind unter anderem Copies (Tabelle für alle Exemplare) und Ledger (Tabelle der Ausleiheinträge). Anhand Österreichischer Statistiken (<https://www.bvoe.at/oeffentliche-bibliotheken/statistik-und-leistungsdaten>), wurde eine Anzahl von 100.000 (10% aller Exemplare in Salzburger Büchereien) Exemplaren bemessen und 274 128 verleihe Pro Jahr (wieder 10% aller verleihe in Salzburg).

Die Größe einer Zeile in Copies beträgt ungefähr 12B (Barcode) + 5B (Media) + 4B (Comp) + 6B (Shelf) + 3B (Loc) + 3B (Header) = 33 Bytes, daraus berechnet sich eine initiale Größe von 4Mbytes (100.000*33B + Puffer). Es wird nicht erwartet, dass sich existierende Kopien verändern, deshalb wurde PCTFREE auf 5% gesetzt. Wird mehr Speicher angefordert, soll für 10% des Ausgangsspeicher angefragt werden.

```
ALTER TABLE "FHS52423"."COPIES"
PCTFREE 5
INITIAL 4M
STORAGE ( NEXT 512K);
```

Die Größe einer Zeile im Ledger beträgt ungefähr 21B (3x Date) + 6B (2x Cost) + 22B (IDs & Barcode) + 3B (Header) = 52 Bytes. Daraus folgt eine initiale Größe von 64M Bytes (Für drei Jahre gedacht 3*274 128*Zeilengröße + Puffer). Next wurde bemessen mit den den beschriebenen 274 128 verleihen pro Jahr (Verleihe * 52B=>16MBytes)

```
ALTER TABLE "FHS52423"."LEDGER"
PCTFREE 25
INITIAL 64M
STORAGE (NEXT 16M);
```


Bei den Zwischentabellen (Index to Index mapping) wird keine Vergrößerung durch Updates erwartet.

```
ALTER TABLE "FHS52423"."MEDIA_GENRE" PCTFREE 0;
ALTER TABLE "FHS52423"."WRITTEN_BY" PCTFREE 0;
```

12 Zugriffspläne analysieren und optimieren

Die meisten Zugriffspläne hatten bereits geringe Kosten und wurden deshalb nicht verändert. Als Beispiel dafür der FSK Check mit Gesamtkosten von 2:

```
# Verleihe & Verfügbarkeit
fsk_check: "SELECT * FROM MEDIAS m
WHERE m.MEDIA_ID = :media_id
AND m.age_restriction <= FLOOR(MONTHS_BETWEEN(SYSDATE, (SELECT birth_date FROM CUSTOMERS WHERE customer_id = :c_id)) / 12)"
```

Operation	Object	Optimizer	Cost	Cardinality	Bytes
SELECT STATEMENT		ALL_ROWS	2	1	44
✓ TABLE ACCESS (BY INDEX MEDIAS)		ANALYZED	1	1	44
INDEX (UNIQUE SCAN MEDIAS_PK)		ANALYZED	0	1	0
✓ TABLE ACCESS (BY INDEX CUSTOMERS)		ANALYZED	1	1	11
INDEX (UNIQUE SC CUSTOMERS_PK)		ANALYZED	0	1	0

Der Query mit den meisten Kosten war calculate_cost_save_in_ledger. Zur Optimierung wurden 2 neue Indizes angelegt, die das Durchsuchen der Tabellen beschleunigt. Dadurch konnten die Kosten von 9 auf 7 gesenkt werden.

```
calculate_cost_save_in_ledger: "
UPDATE LEDGER l
SET l.TOTAL_COST = GREATEST(
0,
ROUND(
(SYSDATE - l.END_DATE) * (
SELECT cost
FROM (
SELECT p.cost
FROM PRICES p
JOIN MEDIA_GENRE g
ON g.GENRES_GENRE_ID = p.GENRES_GENRE_ID
JOIN MEDIAS m
ON m.MEDIA_TYPES_TYPE_ID = p.MEDIA_TYPES_TYPE_ID
AND g.GENRES_GENRE_ID = p.GENRES_GENRE_ID
JOIN COPIES c
ON c.MEDIAS_MEDIA_ID = m.MEDIA_ID
WHERE c.BARCODE_ID = l.COPIES_BARCODE_ID
AND p.VALID <= SYSDATE
ORDER BY p.VALID DESC
)
WHERE ROWNUM = 1
),
2
)
)
WHERE l.COPIES_BARCODE_ID = :barcode
AND l.RETURN_DTIME IS NULL
"
```

```
--Neue Indexe: CREATE INDEX idx_ledger_update_perf
-- ON LEDGER (COPIES_BARCODE_ID, RETURN_DTIME);
-- CREATE INDEX idx_prices_perf
-- ON PRICES (GENRES_GENRE_ID, MEDIA_TYPES_TYPE_ID, VALID_DESC, COST);
```

UPDATE STATEMENT	ALL_ROWS	7	1	40
UPDATE	LEDGER	6	0	0
TABLE ACCESS (BY INDEX ROWID BATCHED)	LEDGER	1	1	40
INDEX (RANGE SCAN)	IDX_LEDGER_UPDATE	1	1	0
COUNT (STOPKEY)		5	0	0
VIEW		5	1	13
SORT (ORDER BY STOPKEY)		5	1	34
NESTED LOOPS		4	1	34
NESTED LOOPS		3	1	31
NESTED LOOPS		2	1	20
INDEX (RANGE SCAN)		1	1	14
TABLE ACCESS (BY INDEX ROWID BATCHED)	MEDIAS	1	1	6
INDEX (UNIQUE SCAN)	MEDIAS_PK	0	1	0
INDEX (FULL SCAN)		1	1	11
INDEX (RANGE SCAN)		1	1	3

Mit demselben Vorgehen konnten auch beim Query statistic_update_on_return die Kosten um 2 mit einem Composit Index reduziert werden.

```
-- CREATE INDEX idx_ledger_stats_helper
-- ON LEDGER (COPIES_BARCODE_ID, RETURN_DTIME DESC, START_DATE);
statistic_update_on_return: "UPDATE STATISTICS s
SET number_time_borrowed = number_time_borrowed + 1,
sum_time_borrowed = sum_time_borrowed + (
SELECT (SYSDATE - START_DATE)
FROM LEDGER
WHERE COPIES_BARCODE_ID = :barcode
-- Instead of checking equal to SYSDATE,
-- just find the most recent record that was just closed.
AND RETURN_DTIME IS NOT NULL
ORDER BY RETURN_DTIME DESC
FETCH FIRST 1 ROW ONLY
)
WHERE COPIES_BARCODE_ID = :barcode"
```

Operation	Object	Optimizer	Cost	Cardinality	Bytes
UPDATE STATEMENT		ALL_ROWS	4	1	18
UPDATE	STATISTICS		3	0	0
TABLE ACCESS (BY INDEX ROWID BATCHED)	STATISTICS	ANALYZED	2	1	18
INDEX (RANGE SCAN)	STATISTICS_IDX	ANALYZED	1	1	0
VIEW			1	1	19
WINDOW (BUFFER PUSHED RANK)			1	1	27
INDEX (RANGE SCAN)		ANALYZED	1	1	27

Bei search_media_genre_type_title konnten die Kosten von 6 nicht weiter gesenkt werden, weil ein full-table-scan notwendig ist.



--Full table access, aber wird gebraucht, weil alle medien durchsucht werden müssen

search_media_genre_type_title: "SELECT m.*, g.CATEGORY, mt.TYPE

FROM MEDIAS m

JOIN MEDIA_TYPES mt ON m.MEDIA_TYPES_TYPE_ID = mt.TYPE_ID

JOIN MEDIA_GENRE mg ON m.MEDIA_ID = mg.MEDIAS_MEDIA_ID

JOIN GENRES g ON mg.GENRES_GENRE_ID = g.GENRE_ID

WHERE LOWER(m.TITLE) LIKE :title OR LOWER(g.CATEGORY) = :genre OR LOWER(mt.TYPE) = :type"

Operation	Object	Optimizer	Cost	Cardinality	Bytes
✓ SELECT STATEMENT		ALL_ROWS	6	1	72
✓ NESTED LOOPS			6	1	72
✓ NESTED LOOPS			6	1	72
✓ HASH JOIN			5	1	28
✓ NESTED LOOPS			5	1	28
✓ STATISTICS C			4	0	0
✓ MERGE JO			4	1	16
TABLE MEDIA_TYPES		ANALYZED	3	1	10
✓ BUFFER			1	6	36
IND MEDIA_GENRE_...		ANALYZED	1	6	36
✓ TABLE ACCE: GENRES		ANALYZED	1	1	12
INDEX (UN GENRES_PK		ANALYZED	0	1	0
TABLE ACCESS (GENRES		ANALYZED	1	1	12
INDEX (UNIQUE SC. MEDIAS_PK		ANALYZED	0	1	0
TABLE ACCESS (BY INI MEDIAS		ANALYZED	1	1	44

```

● INSERT INTO LEDGER (
  START_DATE,
  END_DATE,
  TOTAL_COST,
  COST_PAID,
  CUSTOMERS_CUSTOMER_ID,
  COPIES_BARCODE_ID,
  COPIES_MEDIA_ID)
SELECT
  TRUNC(SYSDATE),
  TRUNC(SYSDATE) + 21,
  (SELECT MAX(COST) FROM PRICES WHERE CREATION_DATE = (
    SELECT MAX(CREATION_DATE)
    FROM PRICES)
  ),
  0,
  :c_id,
  :barcode,
  c.MEDIAS_MEDIA_ID
FROM COPIES c
WHERE c.BARCODE_ID = :barcode
;

```

Operation	Object	Optimizer	Cost	Cardinality	Bytes
INSERT STATEMENT		ALL_ROWS	4	1	14
LOAD TABLE CONVENTIONAL	LEDGER		3	0	0
SORT (AGGREGATE)			2	1	12
TABLE ACCESS (BY INDEX ROWID BATCHED)	PRICES	ANALYZED	2	1	12
INDEX (SKIP SCAN)	PRICES PK	ANALYZED	1	1	0
SORT (AGGREGATE)			1	1	8
INDEX (FULL SCAN)	PRICES PK	ANALYZED	1	7	56
INDEX (FULL SCAN)		ANALYZED	1	1	14

Es wurde ein PRICES_PERF_INDEX (für Sortierte, aktuelle Kosten), dadurch konnte der SKIP SCAN zu einem RANGE SCAN gemacht werden.

Operation	Object	Optimizer	Cost	Cardinality	Bytes
INSERT STATEMENT		ALL_ROWS	3	1	14
LOAD TABLE CONVENTIONAL	LEDGER		2	0	0
SORT (AGGREGATE)			1	1	12
INDEX (RANGE SCAN)		ANALYZED	1	1	12
SORT (AGGREGATE)			1	1	8
INDEX (FULL SCAN)		ANALYZED	1	7	56
INDEX (FULL SCAN)		ANALYZED	1	1	14

```

SELECT * FROM RESERVATIONS
WHERE RESERVATION_ID = (
  SELECT RESERVATION_ID FROM (
    SELECT RESERVATION_ID FROM RESERVATIONS
    WHERE MEDIA_ID = :media_id AND STATUS = 'READY'
    ORDER BY TIME_RESERVATION ASC
  )
  WHERE ROWNUM = 1
)
FOR UPDATE SKIP LOCKED;

```

Operation	Object	Optimizer	Cost	Cardinality	Bytes
SELECT STATEMENT		ALL_ROWS	5	1	25
FOR UPDATE			2	0	0
TABLE ACCESS (BY INDEX ROWID BATCHED)	RESERVATIONS	ANALYZED	2	1	25
INDEX (RANGE SCAN)	RESERVATIONS PK	ANALYZED	1	1	0
COUNT (STOPKEY)			3	0	0
VIEW			3	1	13
SORT (ORDER BY STOPKEY)			3	1	21
TABLE ACCESS (BY INDEX ROWID)	RESERVATIONS	ANALYZED	2	1	21
INDEX (SKIP SCAN)	RESERVATIONS PK	ANALYZED	1	2	0

RESERVATIONS_QUEUE_IDX führt zu einem RANGE SCAN und eliminiert SORT(ORDER BY STOPKEY).

```

UPDATE RESERVATIONS
SET STATUS = 'DONE',
    START_DATE = TRUNC(SYSDATE),
    BARCODE_ID = :barcode
WHERE RESERVATION_ID = :res_id and STATUS = 'READY' and CUSTOMER_ID = :c_id and MEDIA_ID= (SELECT MEDIA_ID FROM COPIES WHERE BARCODE_ID = :barcode);

```

Operation	Object	Optimizer	Cost	Cardinality	Bytes
UPDATE STATEMENT		ALL_ROWS	3	1	25
UPDATE	RESERVATIONS		2	0	0
TABLE ACCESS (BY INDEX ROWID BATCHED)	RESERVATIONS	ANALYZED	2	1	25
INDEX (RANGE SCAN)	RESERVATIONS PK	ANALYZED	1	1	0
INDEX (FULL SCAN)		ANALYZED	1	1	11

Die Subquery für die Media_ID wurde eliminiert, indem das Programm :media_id übergibt.

UPDATE STATEMENT		ALL_ROWS	1	1	25
UPDATE	RESERVATIONS		1	0	0
TABLE ACCESS (BY INDEX ROWID)	RESERVATIONS	ANALYZED	1	1	25
INDEX (UNIQUE SCAN)	RESERVATIONS PK	ANALYZED	0	1	0

Indizes:

 FH Salzburg	Fachhochschule Salzburg
	Fachrichtung: Informationstechnik & System-Management

```

CREATE INDEX idx_ledger_update_perf
ON LEDGER (COPIES_BARCODE_ID, RETURN_DTIME);

CREATE INDEX idx_prices_perf
ON PRICES (GENRES_GENRE_ID, MEDIA_TYPES_TYPE_ID, VALID DESC, COST);

CREATE INDEX idx_ledger_stats_helper
ON LEDGER (COPIES_BARCODE_ID, RETURN_DTIME DESC, START_DATE);

CREATE INDEX PRICES_PERF_IDX ON "FHS52423"."PRICES" ("CREATION_DATE" DESC, "COST" DESC);

CREATE INDEX RESERVATIONS_QUEUE_IDX
ON "FHS52423"."RESERVATIONS" ("MEDIA_ID", "STATUS", "TIME_RESERVATION" ASC);

```

Die DDL-Statements, die zur Analyse verwendet wurden und die stattgefundenen Änderungen sind hier zu sehen:

https://github.com/LuxAlexander/Laboruebungen/blob/main/Datenbanksysteme/Scripte/Script-4_Optimizations_DDL.sql